Content Delivery Network

Distributed Systems

Krish Khimasia Thoyajakshi Gorantla Tanishq Prasad

March 30, 2025

Abstract

The Internet is inherently decentralised, which has been a key factor in its rapid growth and evolution. While this decentralisation has allowed it to scale efficiently, it has also made it challenging to guarantee consistent performance and systematically address network inefficiencies. With the growing demand for high-speed web services delivering various types of multimedia content, which is straining network bandwidth and server capacity, a Content Distribution Network (CDN) serves as an effective solution to enhance Internet service quality.

CDNs have emerged as a crucial solution to address these scalability and performance challenges. By leveraging a network of geographically distributed servers, CDNs cache and serve content closer to users, reducing latency and alleviating the load on origin servers. This approach enhances web performance, ensuring faster and more reliable content delivery. CDNs are widely utilized for various applications, including website acceleration, multimedia streaming, software distribution, and other bandwidth-intensive services.

This paper provides an overview of CDNs and examines their key features. It also discusses the fundamental design considerations and essential design challenges within a CDN system. Furthermore, we explore CDN architectures and analyze three practical implementations, Akamai, AWS CloudFront, and Microsoft Azure Front Door, focusing on their network structures, caching strategies, and performance optimizations.

Through this analysis, we aim to provide a comprehensive understanding of CDN technology, its impact on content delivery, and the trade-offs involved in different implementations.

Preface

The purpose of this report is to study and compare major Content Delivery Networks (CDNs) in use today. Initially, the scope was to explore three prominent CDN solutions. However, due to the limited availability of technical documentation and publicly accessible design papers for some providers we extended our research to cover four.

Among the CDNs reviewed, **Akamai** stood out as the most extensively documented and well-studied. This allowed us to perform a deeper analysis into its internal workings, particularly focusing on how it delivers different forms of content efficiently. For the other three CDNs, i.e. **Azure Front Door**, **Cloudflare**, and **Amazon CloudFront**, we provide focused overviews, highlighting key architectural choices, feature sets, and distinguishing characteristics that set them apart. All three members have contributed to the Akamai section. Tanishq, Krish, and Thoyajakshi have researched on Azure Front Door, Cloudflare, and Amazon CloudFront, respectively.

The report is structured to first introduce the general architecture and goals of CDNs. We then examine the specific design patterns, capabilities, and strategic trade-offs made by each CDN provider.

Contents

Preface **2**

1 Introduction **5**

2 Motivation and Need Behind CDNs **5**

3 General CDN Design[10][8] **6**
3.1 Architecture 7
3.1.1 Components 7
3.1.2 Distribution and Routing 7

4 Common Features and Design Choices[11] **8**

5 Akamai[10] **9**
5.1 The Overlayed Network 9
5.2 Global Monitoring Infrastructure – “The Agents” 9
5.3 Streaming 9
5.4 Tiered Distribution 10
5.5 Edge Computing 10
5.6 Platforms 10
5.6.1 Metadata and Control at the Edge Server 10
5.6.2 Mapping System 11
5.6.3 Communication and Control System 11
5.7 Data Collection and Analysis 11

6 Cloudflare[6] **12**
6.1 Overview of Cloudflare Architecture 12
6.2 DNS Control and Request Handling at the Edge 12
6.3 Data Logging and Analytics at Scale 13
6.4 Performance and Optimization Features[3] 13
6.4.1 File Size Optimization 13
6.4.2 Delivery Enhancements 13
6.4.3 Caching Features 13
6.4.4 Mobile Redirect 13
6.5 Developer Tools and Customization 14
6.6 Security Capabilities 14

7 Azure Front Door[7][9] **15**
7.1 Factors Leading Up to Its Development 15

7.2 Azure Front Door: A Layer-7 Global Load Balancer 15
7.3 Key Features of Azure Front Door 15
7.4 An Azure Front Door Instance 16
7.4.1 Origin Groups 16
7.4.2 Endpoints and Custom Domains 16
7.4.3 Routes and Rules 17
7.4.4 Monitoring and Diagnostics 17
7.4.5 Web Application Firewall and Security 18

8 Amazon CloudFront[1][2] 19
8.1 Overview of CloudFront Architecture 19
8.2 Key Features and Performance Optimizations 19
8.3 Setting Up a CloudFront Distribution 20
8.4 Cache Behaviors and Origin Failover 21
8.5 Monitoring and Observability Tools 21
8.6 Security: AWS WAF and DDoS Protection 21

1 Introduction

The internet is more accessible than ever, with billions of users worldwide connected through broadband, mobile networks, and satellite technology. All these users are consuming online content daily at a scale like never before. The decentralized nature of the internet and the increasing richness of the content consumed make it difficult to provide performance guarantees. Latency of as short as one second seems to be annoying for the user today, hence, efforts to mitigate unpredictable performance in content delivery can provide great incentive.

A Content Delivery Network (CDN) is one such effort that has proved to be effective. The main concept behind CDN is to make content available to the servers at the edge of the user network. This is achieved by using a hierarchy of "replica" servers and cache servers provided by the content provider. The content provider mirrors content in the "origin" server to the "replica" servers all over the world. When a user requests some piece of content, the replica server closest to the user serves the request rather than the origin server, leading to faster response times.

2 Motivation and Need Behind CDNs

In today's digital ecosystem, user expectations for high availability and seamless experiences have pushed the boundaries of what traditional web infrastructure can deliver. While the Internet has evolved significantly, its underlying architecture still presents fundamental limitations for performance-sensitive applications. Factors like routing inefficiencies, congestion at internetwork peering points, and geographic latency issues create major bottlenecks, especially for dynamic and bandwidth-heavy content such as video streaming.

Several high-profile disruptions underscore the fragility of traditional delivery paths. For instance, undersea cable outages in Southeast Asia and the Middle East have previously led to major service disruptions, impacting access across entire regions for days. Business disagreements between ISPs have caused abrupt service partitions, while routing misconfigurations have triggered global blackouts, such as the well-known YouTube hijack incident.

Content Delivery Networks (CDNs) emerged to bridge these infrastructural gaps. Initially designed to cache static web resources near end users, modern CDNs have evolved into highly sophisticated systems capable of dynamically serving entire applications and high-definition video. They aim to minimize the reliance on long-distance Internet communication, reduce latency, improve throughput, and provide better fault tolerance. As streaming video becomes the predominant form of web content—with projections suggesting it will comprise over 90% of global traffic—CDNs play a crucial role in delivering scalable, reliable, and fast media.

3 General CDN Design[10][8]

The fundamental architectural shift that CDNs embody is the decentralization of content delivery. Rather than routing every request back to a few origin servers, CDNs deploy thousands of edge nodes globally. These nodes cache and serve content closer to users, dramatically reducing latency and dependence on congested mid-network links.

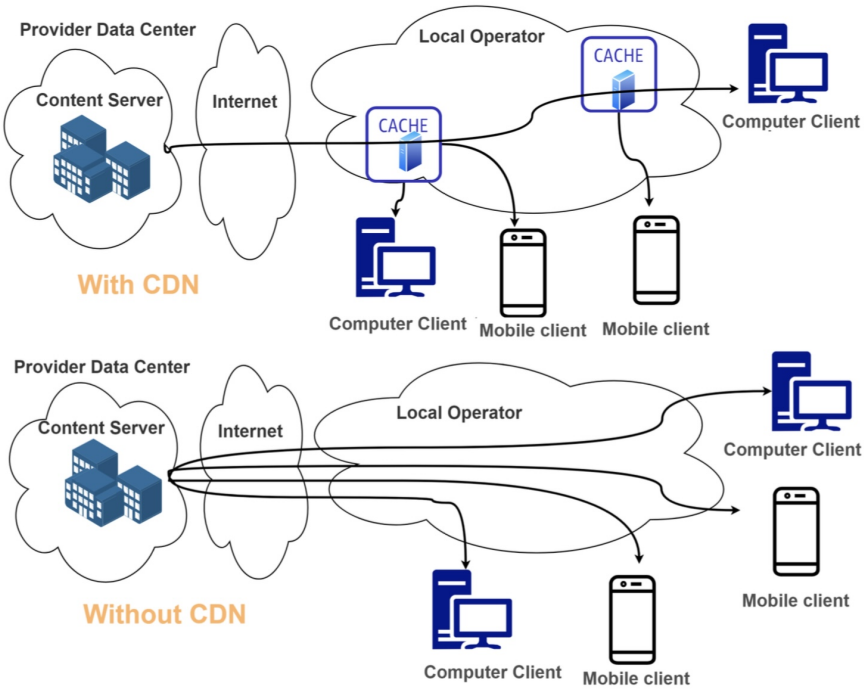

Figure 1: Information flow with and without CDN.

Source: Content Delivery Networks: State of the Art, Trends, and Future Roadmap[8]

A key architectural philosophy in modern CDN design is prioritizing the edge. Instead of focusing solely on fortifying centralized data centers, more emphasis is placed on distributing lightweight server clusters at the "last mile", within ISPs and regional data hubs. This helps avoid traditional bottlenecks and improves both redundancy and user experience.

The overall architecture often involves:

- A mapping system that intelligently routes users to the nearest or most optimal edge server.
- Edge servers which cache, compute, or dynamically generate responses.
- A transport system that connects origin infrastructure to edge nodes, using advanced path optimization and multi-hop strategies.
- Control and analytics layers for configuration management, monitoring, and fine-grained traffic regulation.

3.1 Architecture

3.1.1 Components

Client, **Origin Servers**, **Replica Servers**, **Distribution System**, **Request Routing System**, **Accounting System**, **Billing Organisation**

3.1.2 Distribution and Routing

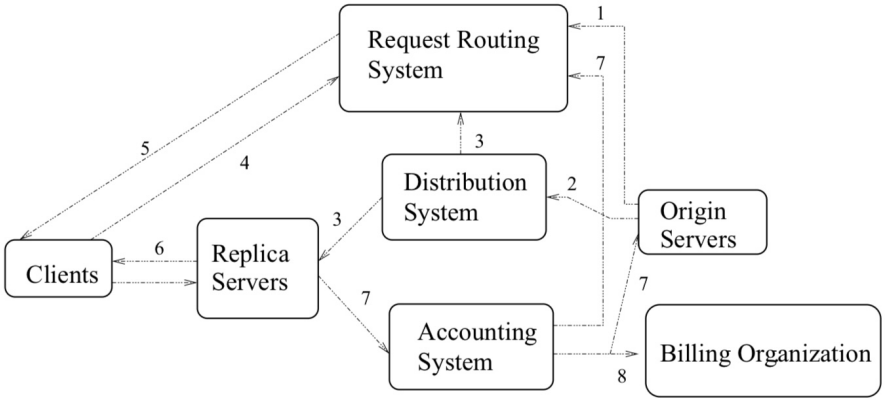

Figure 2: An example image showing CDN architecture.

Source: CDN by Gang Peng[11]

1. The origin server assigns its URI namespace for document objects to the request routing system, enabling their distribution and delivery by the CDN.
2. The origin server gives the content that is to be distributed by the CDN to the distribution system.
3. The distribution system transfers the content to replica servers and works with the request routing system to help determine the appropriate replica server for a client request.
4. The client initiates requests for documents from what it assumes to be the origin server. However, due to URI namespace delegation, the request is redirected to the request routing system.
5. The request routing system directs the client request to an appropriate replica server within the CDN.
6. The corresponding replica server delivers the requested content to the client. Furthermore, the replica server provides the accounting system with data on the content it has delivered.

7. The accounting system compiles and refines accounting data into statistics and content detail records, which are utilized by the origin server and billing organization. Additionally, these statistics serve as feedback for the request routing system.

8. The billing organization uses the records saved to settle with the parties involved in the content distribution and delivery process.

Although peer-to-peer delivery systems offer decentralized scalability, they lack critical enterprise-grade features such as content control, security policies, and analytics. Enterprises require predictable performance, audit trails, and content governance, where CDNs provide superior solutions.

4 Common Features and Design Choices[11]

Modern CDN platforms are built on a few key principles that ensure performance and resilience at Internet scale:

1. **Failure Resilience**: Failures are treated as expected, not exceptional. The system is designed to gracefully tolerate outages at server, cluster, or network levels through redundancy and self-healing mechanisms.

2. **Scalability**: All components are designed to scale elastically, whether by accommodating new edge nodes or dynamically redistributing load.

3. **Autonomy**: Human intervention is minimized through automation in monitoring, updates, and routing adjustments.

4. **Performance**: Optimizations are built at both the protocol level (e.g., TCP tuning) and application layer (e.g., prefetching content or compressing responses).

One critical feature CDNs provide is fine-grained control to enterprises. Using metadata configurations, organizations can dictate how different classes of content should be handled. They can define caching policies, header transformations, failover strategies, and even embed business logic directly at the edge. This allows customization of behavior based on path, user location, bandwidth, or even device type.

By supporting a high degree of configurability, CDNs empower content providers to treat the CDN infrastructure as an extension of their own systems, offering both control and observability at scale.

5 Akamai[10]

5.1 The Overlayed Network

Akamai's platform operates on a software-based virtual network layered over the Internet, designed to sidestep inefficiencies in native routing and infrastructure. By positioning servers deep within ISP networks across the globe, it bypasses middle-mile congestion and latency issues. This overlay network dynamically adapts to Internet conditions, ensuring seamless delivery performance for web applications, content, and media streams.

5.2 Global Monitoring Infrastructure – "The Agents"

To maintain high quality and availability, Akamai deploys numerous agents worldwide that actively simulate user interactions. These agents measure parameters like stream startup times, playback quality, and network latency, feeding real-time insights into Akamai's internal systems for routing and optimization. This user-centric monitoring enables the platform to react to issues proactively.

5.3 Streaming

- **Metrics Developed:** Akamai defines performance through user-experience metrics like stream availability, startup latency, smooth playback (interruption rates), and bitrate quality. These help quantify how well the service performs in diverse environments.
- **The Overlay Network for Streaming:** Streaming data uses a specialised overlay that operates differently from the traditional delivery network. It is structured in a publish-subscribe model and optimized for real-time traffic.
- **Reflectors:** Reflectors act as scalable intermediaries that take streams from entrypoints and distribute them to edge clusters. They help replicate data to multiple destinations, supporting high traffic volumes and ensuring resilience through alternate delivery paths.
- **Path Optimization:** Akamai employs multiple redundant paths and chooses optimal ones based on real-time network feedback. If one path degrades, the system reroutes traffic automatically to maintain performance.
- **Prebursting:** To reduce stream startup times, Akamai introduces prebursting, sending an initial burst of data to allow quick buffering before playback begins. This technique minimizes perceived delay.

5.4 Tiered Distribution

- **Transport System:** To reduce dependency on origin servers, Akamai leverages parent clusters. These intermediate nodes serve requests for uncached content, minimizing origin load and enhancing performance.
- **Path Optimization for Tiered Delivery:** Akamai routes traffic through optimized paths using overlay intelligence. This allows efficient content flow from origin to parent and finally to edge nodes.
- **Packet Loss Reduction:** By using disjoint paths and forward error correction, Akamai significantly reduces packet loss, even on congested networks, without noticeable delay increases.
- **Own Protocol:** Internally, Akamai servers communicate over a proprietary transport protocol. It improves on standard TCP by utilizing persistent connections, adaptive retransmission, and smarter window sizing.
- **Application Optimizations (Asynchronous Prefetch):** Akamai servers can analyze and prefetch embedded resources when delivering web pages. This asynchronous fetch reduces perceived load times and enhances responsiveness, even for dynamic or uncacheable assets.

.

5.5 Edge Computing

- **Challenges:** Distributing application logic to the edge introduces issues like session state management, fault resilience, and security isolation. These are made complex by the scale and geographic spread of the edge infrastructure.
- **Features:** EdgeComputing allows customer applications to execute at the edge. It supports request-driven Java apps, XSLT transformations, content assembly, and conditional logic based on user attributes.
- **Use Cases:** Examples include polling systems, product search, lightweight data entry forms, and dynamic page generation. Some apps can run fully at the edge, while others offload the presentation layer for responsiveness.

5.6 Platforms

5.6.1 Metadata and Control at the Edge Server

- **Metadata Configuration:** Content handling on edge servers is controlled via metadata rules. These configurations dictate how content is cached, transformed, or redirected

based on request parameters.

- **Functionalities Provided:** Edge servers can manipulate headers, cache dynamic fragments, apply security policies, and optimize TCP behavior. They also support logic-driven actions like fallback to backups upon origin failure.
- **XML and Testing:** Metadata is defined in XML and distributed securely. A multi-phase staging and testing pipeline ensures safe rollouts and automatic rollback if issues are detected.
- **Fault Tolerance:** Edge nodes self-monitor and withdraw from traffic mapping if faults are detected. The mapping system reroutes traffic in real time to ensure continuity.

5.6.2 Mapping System

- **Scoring:** Akamai continuously assesses global network health through pings, traceroutes, and BGP data. It builds topology-aware maps, identifying how well IP blocks connect across the Internet.
- **Mapping:** Users are first mapped to optimal clusters, then to servers within those clusters based on load, cache locality, and network proximity. This ensures efficient delivery and reduces latency.

5.6.3 Communication and Control System

- **Components Communicated and Monitored:** Internal communication spans real-time control messages, key distribution, software configuration, and metadata updates. Akamai uses pub-sub models, secure replication, and RPC over its own overlay to maintain control across 60,000+ nodes.

5.7 Data Collection and Analysis

- **Logs:** Massive volumes of HTTP logs are collected, validated, and processed daily—amounting to over 100TB. These logs inform billing, analytics, and diagnostics.
- **Real-Time Monitoring:** A distributed in-memory database (Query) allows SQL-like real-time monitoring across the platform. It supports instant alerting, state validation, and performance dashboards.
- **Reporting:** Akamai's analytics engine leverages MapReduce-like systems to prepare multi-dimensional insights for customers, covering traffic, user geography, and content performance.

6 Cloudflare[6]

Cloudflare is one of the most prominent names in the Content Delivery Network (CDN) landscape, widely recognized for its robust DDoS mitigation, performance optimization, and security capabilities. Established in 2009, Cloudflare's platform now supports millions of websites globally and has become a critical component of the modern internet infrastructure. As of 2017, it handled nearly 10% of all global internet traffic, a testament to its scale and reliability.

6.1 Overview of Cloudflare Architecture

At the core of Cloudflare's architecture lies its expansive global network, which is engineered to get as close to end users as possible. This is achieved by physically deploying edge servers at Internet Exchange Points (IXPs) where internet service providers (ISPs) interconnect. Cloudflare participates in more internet exchanges than any other CDN provider, ensuring optimal latency and connectivity.
Cloudflare employs Anycast routing, rather than traditional DNS-based routing, to serve user requests from the nearest available data center. This design supports immediate failover, automatic load balancing, and DDoS attack repartitioning, where malicious traffic is absorbed and diffused across the entire global network. The benefits of such a large Anycast infrastructure include proximity to users, improved peering with local ISPs, simplified rate limiting, and strong resilience against large-scale attacks.
One of Cloudflare's intelligent routing technologies is Argo[4], a usage-based service that dynamically selects the fastest and most reliable paths across Cloudflare's network using real-time latency measurements. Paired with tiered caching, Argo helps reduce origin server load and improves delivery efficiency by retrieving content from intermediate cache layers when possible.

6.2 DNS Control and Request Handling at the Edge

Cloudflare not only acts as a CDN but also takes over the role of the authoritative DNS server for client domains. It decides how DNS records are served, enhancing performance and security. This setup involves a two-tier DNS system:

- rrDNS: Cloudflare's internal DNS system, acting as the source of truth.
- External DNS: Services public DNS requests, returning Cloudflare IPs for proxied records.

DNS resolution, including resolving CNAME records, is managed by Cloudflare's DNS Resolver, which ensures fast and accurate resolution. At the edge, all incoming HTTP requests are processed through a lightweight runtime environment written in Lua. This layer handles transformations such as compression (GZIP, Polish), header manipulation, and WAF (Web Application Firewall) filtering. TLS management is handled by a separate service written in Go, ensuring secure connections are maintained across the network.

6.3 Data Logging and Analytics at Scale

Given the volume of traffic Cloudflare manages, it generates massive amounts of log data. This data is streamed to large-scale data centers, where it is aggregated and analyzed using asynchronous queues for stability and scalability. Go microservices perform parallel processing, and the data is consolidated using PostgreSQL with CitusDB extensions. While enterprise customers may access logs in Apache format, data retention is limited due to sheer volume.

6.4 Performance and Optimization Features[3]

Cloudflare offers a rich set of tools to optimize web performance.

6.4.1 File Size Optimization

- **Image Resizing**, **Polish**: Image compression to reduce bandwidth usage.
- **Auto Minify**: Minimizes HTML, CSS, and JavaScript to reduce file size.
- **Brotli Compression**: A modern compression algorithm for HTTPS traffic that improves page load times.

6.4.2 Delivery Enhancements

- **Enhanced HTTP/2 Prioritization**: Improves the loading behavior of assets over HTTP/2.
- **TCP Turbo**: Custom-tuned TCP optimizations to reduce latency and increase throughput.
- **Mirage**: Optimizes image loading for mobile devices, especially on slow connections.
- **Rocket Loader™**: Improves paint time for JavaScript-heavy pages by loading scripts asynchronously.

6.4.3 Caching Features

- **URL Prefetching**: Allows Cloudflare to prefetch assets from headers.
- **Browser Cache Expiration**: Lets site owners control how long browsers cache content.
- **Always Online™**: Serves a cached version of the site when the origin server is unavailable.

6.4.4 Mobile Redirect

- Redirects users to a mobile-optimized version of the site if the original site is not responsive.

6.5 Developer Tools and Customization

Cloudflare provides developers with powerful tools to extend and customize functionality:

- **Cloudflare Workers**[5]: A serverless platform allowing JavaScript to be executed at the edge, ideal for low-latency API responses and custom routing logic.
- **Page Rules**: Offers fine-grained control over CDN behavior based on specific URL patterns.
- **Load Balancing and Video Streaming**: Usage-based products with built-in failover and intelligent performance routing to optimize traffic delivery and media content distribution.

Cloudflare also maintains a growing library of third-party integrations, allowing clients to enhance site functionality through analytics, monitoring, SEO tools, and more.

6.6 Security Capabilities

In addition to WAF and DDoS protection, Cloudflare offers a Scrape Shield, which defends websites from content theft by obscuring source code, preventing hotlinking, and enabling email obfuscation. Combined with TLS, bot management, and threat intelligence, these tools form a comprehensive security perimeter around customer websites.

Cloudflare represents the modern standard of a CDN—combining traditional caching and content delivery with deep security integration, intelligent routing, and edge computing capabilities. Its global presence, innovative features like Argo and Workers, and unmatched scalability position it as not only a CDN but a foundational layer of today's internet. For organizations seeking fast, secure, and customizable content delivery, Cloudflare remains a premier choice.

7 Azure Front Door[7][9]

7.1 Factors Leading Up to Its Development

Modern applications are often deployed across multiple Azure regions and hybrid or multi-cloud environments to achieve high availability and low latency. However, this presents challenges in the efficient routing of user traffic. Traditional solutions like Azure Traffic Manager and Azure Load Balancer fall short: Traffic Manager relies on DNS and work with TTL of DNS records, while Azure Load Balancer operates at Layer 4 and lacks application-level awareness (e.g., URL paths, SSL, cookies). A more advanced solution was needed, combining global reach with real-time traffic management.

7.2 Azure Front Door: A Layer-7 Global Load Balancer

Azure Front Door addresses these limitations by offering:

- A single global endpoint using Anycast
- Latency-based routing with health probes
- Layer-7 features: SSL offload, URL rewrite, caching, compression
- Built-in CDN and Web Application Firewall (WAF)

It unifies global load balancing with application-layer knowledge, enhancing both performance and security. It leverages Microsoft's extensive global infrastructure, comprising numerous edge servers known as Points of Presence (PoPs). These PoPs, which are also used by Azure's Content Delivery Network (CDN), act as the edge locations for Front Door, allowing user requests to be routed via the closest entry point using Anycast. This improves latency and performance by keeping initial connections local. Requests are then forwarded to the configured origin servers, which can be public IP addresses, publicly resolvable DNS names, or, in the Premium SKU, private endpoints within a virtual network.

7.3 Key Features of Azure Front Door

Azure Front Door offers several advanced Layer-7 capabilities that enhance application performance and reliability. One such feature is **Split TCP**, which establishes the TCP and TLS handshake at the nearest edge location instead of the origin server. This reduces latency by avoiding long round trips for session setup and accelerates content delivery.

Additionally, **caching** is supported at the edge, enabling frequently requested content to be served locally without hitting the origin every time. **Compression** reduces payload size, improving load times over slower connections. For session consistency, **cookie-based affinity** allows requests from the same client to be directed to the same backend.

Azure Front Door also supports **URL rewriting and redirection**, which enables modification of request paths or redirection to alternate URLs as needed. These operations can be combined with custom rules based on path, protocol, geography, or headers.

7.4 An Azure Front Door Instance

Each Azure Front Door setup begins with creating an instance that acts as the central configuration point. Being a global service, it does not require a region selection and is designed to manage frontends, routing rules, and backends (called origins).

7.4.1 Origin Groups

Origin groups define the set of backend services that Azure Front Door routes traffic to. Within a group, each origin can be assigned a **priority** and a **weight**. Priority allows configuring failover behaviour, i.e. Front Door will attempt to send traffic to higher-priority origins first, and fall back to lower-priority ones if needed. Weights control load distribution among origins with the same priority, making it possible to gradually shift traffic. This is particularly useful in deployment strategies like canary or ring rollouts, where a new version is tested with a small portion of traffic before increasing its share.

Azure Front Door also uses **health probes** to monitor the availability of each origin. These probes periodically check endpoints (typically via HTTP/HTTPS) to ensure only healthy backends receive traffic, improving fault tolerance and service continuity.

7.4.2 Endpoints and Custom Domains

Azure Front Door allows the creation of multiple front-end **endpoints**, each serving as a globally accessible entry point. These endpoints can be assigned **custom DNS names** instead of relying solely on the default `azurefd.net` domain. Each endpoint can have separate routing rules and origin group associations, enabling support for multiple services or use cases within a single Front Door instance.

To use a custom domain, it must first be validated. Azure Front Door requires the addition of a specific **DNS TXT record** to prove ownership of the DNS zone. This is necessary because validation cannot rely on just checking a CNAME alias, especially when the domain is already in production. Once validated, the domain can be associated with an endpoint, and Azure Front Door will automatically provision and manage an SSL certificate for secure HTTPS access.

The validated custom domain must be explicitly **associated with one or more endpoints** to receive traffic. Each endpoint can then apply its own routing and rules, allowing for flexible and secure traffic control across different services or environments.

7.4.3 Routes and Rules

Routing in Azure Front Door is defined through **routes**, which map frontend requests to backend origin groups based on specific criteria. A route can match on a combination of the **frontend domain**, **protocol** (HTTP or HTTPS), and the **URL path**. This enables **path-based routing**, where different parts of a website (e.g., `/api`, `/static`) can be served by different origin groups. These configurations are managed under the *Endpoint Manager* section of the Azure portal.

Routes also support additional options such as **data compression**, **HTTP to HTTPS redirection**, and **query string caching** behaviors. Query string caching can be adjusted to control whether variations in query parameters affect cache lookup, which is useful for optimizing content delivery.

Further control is available through **rule sets** associated with specific routes. Rules can inspect attributes like the user's **IP address**, **geographic location**, **device type**, or HTTP method (e.g., disallowing PUT requests), and dynamically alter behavior. For example, traffic from a certain region can be routed to a dedicated origin group, or requests from mobile devices can be redirected to a mobile-optimized backend.

All of this is configurable using the rule set editor in the Azure portal, where multiple conditions and actions (such as **URL rewrite**, **redirect**, or **header modification**) can be chained together for advanced traffic control.

7.4.4 Monitoring and Diagnostics

Azure Front Door integrates with Azure's monitoring ecosystem to provide visibility into performance, traffic patterns, and security events. This includes native support for **Azure Monitor**, **Log Analytics**, and **Azure Metrics Explorer**.

Azure Monitor collects performance and usage data, enabling real-time tracking of request volume, latency, backend health, and response status codes. These metrics help detect anomalies, usage spikes, or backend failures.

Log Analytics can be used to run detailed queries against diagnostic logs and telemetry data. Front Door logs include valuable insights such as client IP, request method, URL path, response code, and WAF action (if applicable). This data is especially useful for debugging, auditing, and forensic analysis.

Azure Metrics Explorer provides a visual interface to explore time-series metrics. It allows users to plot trends, set up custom dashboards, and visualize patterns over time.

Together, these tools offer a comprehensive observability stack, helping administrators maintain reliability, enforce SLAs, and respond quickly to incidents.

7.4.5 Web Application Firewall and Security

Azure Front Door integrates with the Azure Web Application Firewall (WAF) to provide robust security at the edge. The WAF operates directly at the **Point of Presence (PoP)** level ensuring that malicious traffic is filtered before it reaches the backend origin servers. This design avoids introducing compute overhead or additional latency on the origin infrastructure.

The WAF offers features such as **rate limiting**, allowing administrators to restrict excessive or suspicious requests from clients. In the Premium SKU, WAF includes pre-configured OWASP rule sets, bot protection, and geo-based access control. These protections, combined with custom routing and rule capabilities, make Azure Front Door a comprehensive platform for secure, scalable, and performant global application delivery.

8 Amazon CloudFront[1][2]

Amazon CloudFront is the content delivery network (CDN) of Amazon Web Services (AWS), launched in 2008, now has 700+ PoPs in 100+ cities across 50+ countries. AWS introduced CloudFront to provide a scalable, pay-as-you-go CDN tightly integrated with other AWS services (like S3 storage and EC2 compute). AWS CloudFront, Amazon's global CDN service, is responsible for an estimated 10-15% of global CDN traffic as of 2024.

By leveraging Amazon's global infrastructure, CloudFront was designed to automatically route users to the nearest AWS edge location, improving download speeds and reliability for distributed audiences. This strategic addition to the AWS ecosystem aimed to make high-performance content delivery more accessible and cost-effective, potentially disrupting traditional CDN pricing models by charging only for usage and eliminating upfront commitments.

8.1 Overview of CloudFront Architecture

When a viewer requests content via CloudFront, DNS and Anycast routing direct the request to the optimal edge location, typically the one with the lowest latency. Each edge location is a cluster of caching servers that holds recently requested content. If the edge already has the object cached, it returns it immediately. Otherwise, it retrieves it from the origin using CloudFront's hierarchical caching and AWS's global backbone network.

A key aspect is the two-tier caching strategy: **Regional Edge Caches (RECs)** sit between edge PoPs and the origin. If an edge cache misses, the request is routed to a REC before going to origin. This improves cache hit ratios and reduces origin load.

All CloudFront nodes use the **AWS private backbone**, ensuring even origin fetches avoid the public internet. This yields high throughput and reliability.

8.2 Key Features and Performance Optimizations

CloudFront provides several optimizations and advanced features to enhance performance:

- **Global Edge Network and Request Routing:** Automatic request routing to nearest edge via Anycast, with origin fetches using AWS's backbone.
- **Support for Static and Dynamic Content:** CloudFront accelerates both static assets (like HTML, CSS, JavaScript, images) and dynamic content, including REST APIs, GraphQL, and even WebSockets. It supports real-time use cases such as live chat, live streaming, and API-driven applications with low latency and high reliability.
- **Modern Protocols:** Supports HTTP/2 and HTTP/3 (QUIC), reducing latency via multiplexing, reducing bandwidth through header compression and fast TLS.

- **Content Caching and Customization:** Fine control with TTLs, cache keys, origin request policies. Supports partial content (range requests), **Origin Shield** for traffic aggregation, and object invalidation.
- **Automatic Compression:** Brotli/gzip support for compressible types, reducing size and improving page speed.
- **Edge Computing:** AWS CloudFront supports edge-based compute capabilities through **Lambda@Edge** and **CloudFront Functions**. *Lambda@Edge* enables dynamic content generation, request/response modification, and user-personalized logic execution at regional edge locations. It is well-suited for complex workflows such as authentication, A/B testing, or dynamic page assembly. *CloudFront Functions*, on the other hand, are optimized for lightweight operations, such as header rewriting, URL normalization, and simple redirects, executed at the global edge for lower latency and higher scalability.
- **AWS Integration:** Works with S3, EC2(Elastic Compute Cloud), ELB, MediaPackage, supports ACM SSL, CloudFormation, CLI, and DevOps pipelines.

8.3 Setting Up a CloudFront Distribution

Setting up an AWS CloudFront distribution typically involves the following steps:

1. **Define Origin(s):** Specify the origin resource, such as an S3 bucket, EC2 instance, on-premise server, or AWS Media Services. Apply appropriate access controls for private content delivery.
2. **Create the Distribution:** Establish the default origin and cache behavior settings.
3. **Configure Cache Behavior:** Define cache key policies, allowed HTTP methods, time-to-live (TTL) values, forwarding rules for headers/cookies/query strings, protocol redirection, and compression settings.
4. **Enable Additional Features:** Integrate with AWS WAF, enable logging, apply geo-restrictions, and choose a pricing class based on edge location tiers.
5. **Domain and SSL:** Use the auto-generated CloudFront domain name or configure a custom domain with ACM-provisioned SSL certificates.
6. **Deploy and Test:** Allow for global edge propagation; validate response headers, caching behavior, and protocol enforcement.
7. **Manage and Monitor:** Use invalidation tools, adjust cache behavior as needed, and monitor metrics via CloudWatch and logging.

8.4 Cache Behaviors and Origin Failover

- **Cache Behaviors:** Control request routing based on URL patterns. Configure selective forwarding of headers, cookies, and query strings. Enforce HTTP or HTTPS, and customize error responses.
- **Origin Failover:** Use origin groups to define primary and secondary origins. Failover is triggered automatically on 5xx responses or timeouts. Retry behavior applies only to idempotent methods (GET, HEAD, OPTIONS).
- **Lambda@Edge in Failover:** Implement custom logic to route traffic to alternate origins dynamically based on request context or error types.

8.5 Monitoring and Observability Tools

- **CloudWatch Metrics:** Track metrics such as request count, HTTP errors, data transferred, cache hit/miss ratios. Use alarms for thresholds or anomaly detection.
- **Logs:** Enable standard access logs to S3 for post-analysis. For real-time observability, stream logs using Amazon Kinesis Data Streams.
- **Reports:** View analytics on usage trends, frequently accessed objects, regional traffic patterns, and top referrers via the AWS console.
- **Integrations:** Integrate with CloudTrail for API call auditing, AWS Config for compliance checks, and third-party tools such as Datadog or Splunk for extended monitoring.

8.6 Security: AWS WAF and DDoS Protection

- **Transport Security:** TLS termination is handled at edge locations. HTTPS enforcement and SSL certificates can be provisioned via AWS Certificate Manager (ACM).
- **DDoS Protection:** AWS Shield Standard provides automatic protection against volumetric attacks. Shield Advanced offers enhanced mitigation capabilities and access to the AWS DDoS Response Team (DRT).
- **Web Application Firewall (WAF):** Use WAF rules to allow, block, or monitor traffic based on IP address, country, HTTP headers, regex patterns, or OWASP rule sets. Real-time inspection and logging are supported.
- **Access Control:** Restrict access using signed URLs or signed cookies. Enforce private S3 access with Origin Access Identity (OAI) or the more modern Origin Access Control (OAC) mechanism.

- **Advanced Security Features:** Use Field-Level Encryption to secure sensitive fields in transit. Geoblocking can restrict content access based on regional licensing or compliance requirements.

In conclusion, Amazon CloudFront offers a globally distributed, feature-rich CDN platform deeply integrated with AWS's ecosystem. It enhances application speed, scalability, and security, making it a foundational choice for modern content delivery.

References

[1] Amazon Web Services. *Amazon CloudFront – How CloudFront Works*. Accessed: March 30, 2025. 2024. URL: https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/Introduction.html.

[2] Amazon Web Services. *Amazon CloudFront Features*. https://aws.amazon.com/cloudfront/features/?whats-new-cloudfront.sort-by=item.additionalFields.postDateTime&whats-new-cloudfront.sort-order=desc. Accessed: March 30, 2025. 2024.

[3] Astral Web Inc. *Cloudflare Introduction (Tutorial for Beginners)*. 2019. URL: https://www.youtube.com/watch?v=7riFvux9Bw4.

[4] Cloudflare, Inc. *Argo Smart Routing: Faster Web Performance*. URL: https://www.cloudflare.com/products/argo-smart-routing/.

[5] Cloudflare, Inc. *Cloudflare Workers Documentation*. URL: https://developers.cloudflare.com/workers/.

[6] Cloudflare, Inc. *Recap: How does Cloudflare work*. 2017. URL: https://www.youtube.com/watch?v=zX0H-WV5CEE.

[7] John Savill's Technical Training. *Microsoft Azure Front Door Deep Dive*. 2021. URL: https://www.youtube.com/watch?v=DHiZbIks9i0.

[8] S. Krishnan and Ramesh K. Sitaraman. "Content Delivery Networks: State of the Art, Trends, and Future". In: *ACM Computing Surveys* 52.5 (2020), pp. 1–34. DOI: 10.1145/3380613. URL: https://dl.acm.org/doi/10.1145/3380613.

[9] Microsoft. *Azure Front Door and CDN documentation*. URL: https://learn.microsoft.com/en-us/azure/frontdoor/.

[10] Erik Nygren, Ramesh K. Sitaraman, and Jim Sun. "The Akamai Network: A Platform for High-Performance Internet Applications". In: *ACM SIGOPS Operating Systems Review* 44.3 (2010), pp. 2–19. DOI: 10.1145/1842733.1842736.

[11] Gang Peng. *CDN: Content Distribution Network*. arXiv preprint cs/0411069. 2004. DOI: 10.48550/arXiv.cs/0411069. URL: https://arxiv.org/abs/cs/0411069.